SLIIT
Discover Your Future

IT1170 – Data Structures and Algorithms

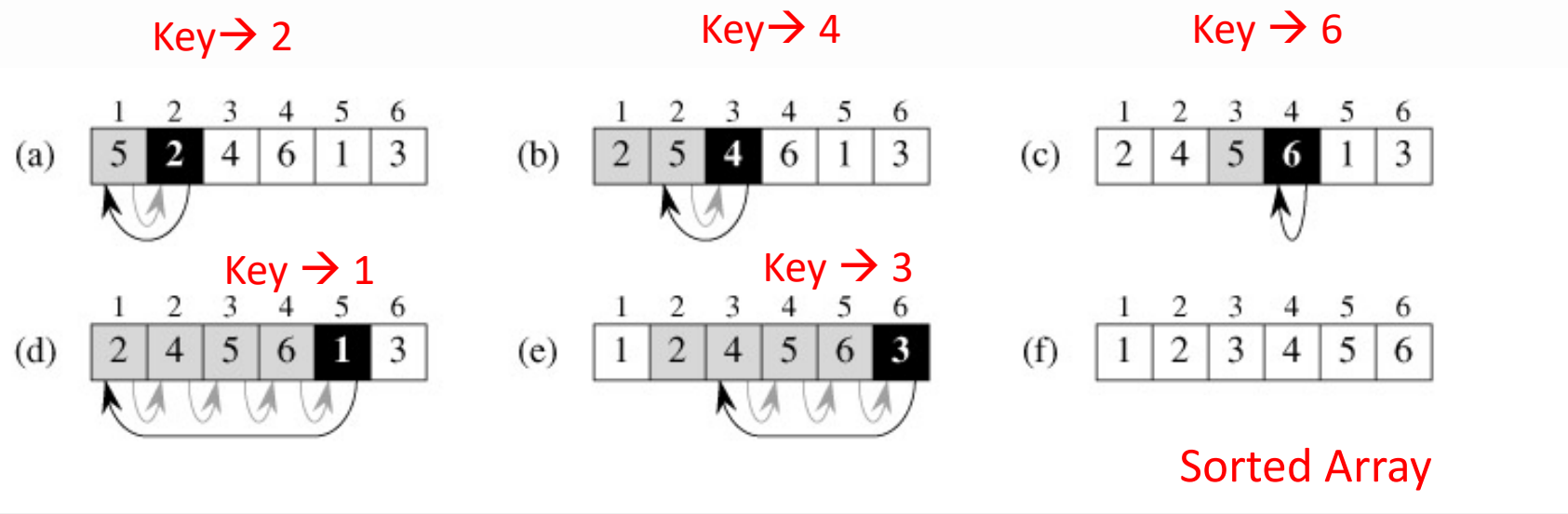
Lecture 02

Insertion Sort and Big O Notation

Recap

- What is an algorithm?
- Properties of an algorithm.
- Design methods.
- Pseudocode.
- Analysis(Operation count & Step count, RAM model).

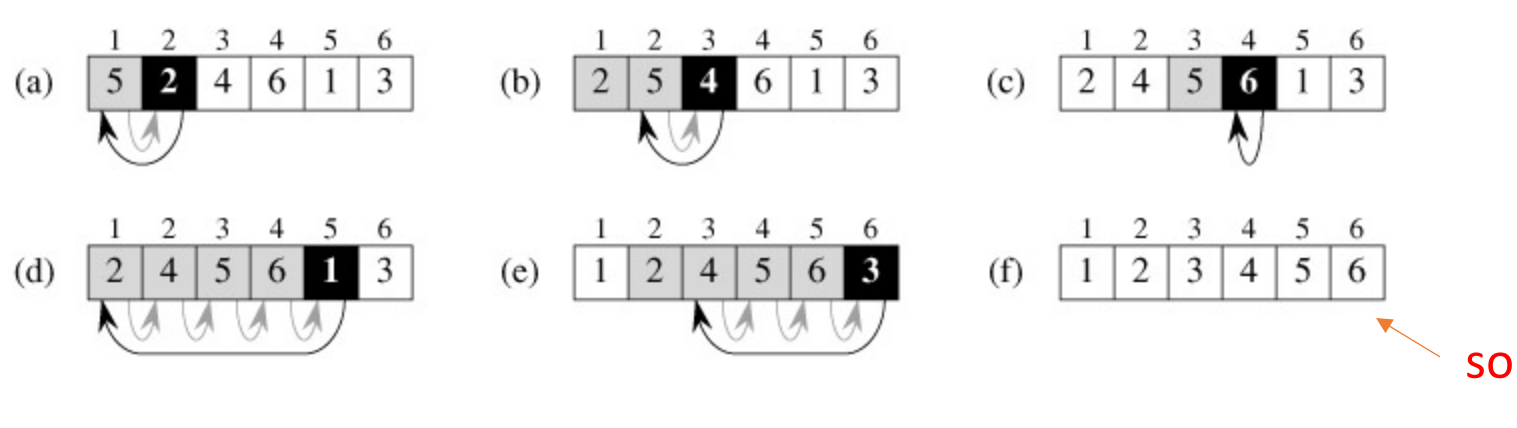
Insertion sort



Pseudocode for insertion sort.

```
INSERTION-SORT(A)
for j = 2 to A.length
        key = A[j]
        // Insert A[j] into the sorted sequence A[1..j-1]
        i = j - 1
        While i > 0 and A[i] > key
                A[i+1] = A[i]
                i = i-1
        A[i+1]  = key
```

Insertion sort - Example



sorted array

- *(a)-(e)* The iterations of the *for* loop → lines 1-8.
- In each iteration, the black rectangle holds the key taken from *A*[*j*],
- Key is compared with the values in shaded rectangles to its left → line 5.
- Shaded arrows show array values moved one position to the right → line 6,
- Black arrows indicate where the key is moved to → line 8.

Exact analysis of Insertion sort

- Time taken for the algorithm will depend on the input size (number of elements of the array)

Running Time (Time complexity):

This is the number of primitive operations or steps executed through an algorithm given a particular input.

Running Time : T(n)

	INSERTION-SORT(A)	Cost	Times
1	**for** j = 2 **to** A.length	c_1	**n**
2	key = A[j]	c_2	**n-1**
3	// Insert A[j] into the sorted // sequence A[1..j-1]	**0**	**n-1**
4	i = j – 1	c_4	**n-1**
5	**While** i > 0 **and** A[i] > key	c_5	$\sum^{n}_{j=2} t_j$
6	A[i+1] = A[i]	c_6	$\sum^{n}_{j=2} (t_j - 1)$
7	i = i-1	c_7	$\sum^{n}_{j=2} (t_j - 1)$
8	A[i+1] = key	c_8	**n-1**

i^{th} line takes time c_i where c_i is a constant.

For each j=2,3,…,n , t_j be the number of times the while loop is executed for that value of j

Running Time(contd.)

$$T(n) = c_1 n + c_2 (n-1) + c_4 (n-1) + c_5 \sum_{j=2}^{n} t_j$$

$$+ c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1)$$

- Best Case (Array is in sorted order)
  - $T(n) \rightarrow an+b$
- Worst Case (Array is in reverse sorted order)
  - $T(n) \rightarrow cn^2 + dn + e$

Worst Case $T(n) \rightarrow cn^2 + dn + e$

Worst case: The array is in reverse sorted order.

- Always find that $A[i] > key$ in while loop test.
- Have to compare *key* with all elements to the left of the jth position $\Rightarrow$ compare with $j - 1$ elements.
- Since the while loop exits because i reaches 0, there's one additional test after the $j - 1$ tests $\Rightarrow t_j = j$.
- $\sum_{j=2}^{n} t_j = \sum_{j=2}^{n} j$ and $\sum_{j=2}^{n} (t_j - 1) = \sum_{j=2}^{n} (j - 1)$.
- $\sum_{j=1}^{n} j$ is known as an ***arithmetic series***, and equation (A.1) shows that it equals $\frac{n(n+1)}{2}$.

Worst Case $T(n) \rightarrow cn^2 + dn + e$

- Since $\sum_{j=2}^{n} j = \left(\sum_{j=1}^{n} j\right) - 1$, it equals $\frac{n(n+1)}{2} - 1$.
  [The parentheses around the summation are not strictly necessary. They are there for clarity, but it might be a good idea to remind the students that the meaning of the expression would be the same even without the parentheses.]
- Letting $k = j - 1$, we see that $\sum_{j=2}^{n}(j-1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}$.
- Running time is

$$
\begin{aligned}
T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\
&\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) \\
&= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right) n \\
&\quad - (c_2 + c_4 + c_5 + c_8)\,.
\end{aligned}
$$

- Can express $T(n)$ as $an^2 + bn + c$ for constants a, b, c (that again depend on statement costs) $\Rightarrow T(n)$ is a *quadratic function* of n.

Asymptotic Notations

- RAM Model has some problems.
- Exact analysis is very complicated.

Therefore we move to **asymptotic notation**

- Here we focus on determining the biggest term in the complexity function.
- Sufficiently large size of n.

Asymptotic Notations(Contd.)

- There are three notations.

O - Notation

Θ - Notation

Ω - Notation

Big O - Notation

- Introduced by Paul Bechman in 1892.
- We use Big O-notation to give an upper bound on a function.

Definition:

$O(g(n)) = \{ f(n)$: there exist positive constants c and n_o such that

$0 \leq f(n) \leq cg(n)$ for all $n \geq n_o\}$.

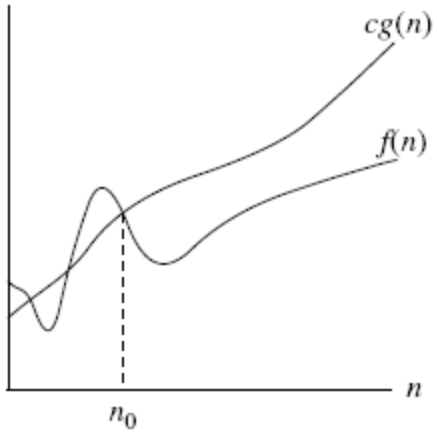
Eg: What is the big O value of f(n)=2n + 6 ?

$c = 4$

$n_o = 3$

g(n)=n therefore

$f(n) = O(n)$



$g(n)$ is an ***asymptotic upper bound*** for $f(n)$.
If $f(n) \in O(g(n))$, we write $f(n) = O(g(n))$

Back to the example

- Alternative calculation:

	cost	times
sum = 0	c_1	1
for i = 1 to n	c_2	n+1
sum = sum + A[i]	c_3	n

$$T(n) = c_1 + c_2\,(n+1) + c_3\,n$$

$$= (c_1 + c_2) + (c_2 + c_3)\,n$$

$$= c_4 + c_5\,n \quad \rightarrow O(n)$$

Proof: $c_4 + c_5\,n \le c\,n \rightarrow$ TRUE for $n \ge 1$ and $c \ge c_4 + c_5$

Big O – Notation(Contd.)

Assignment (s = 1)

Addition (s+1) O(1)

Multiplication (s*2)

Comparison (S<10)

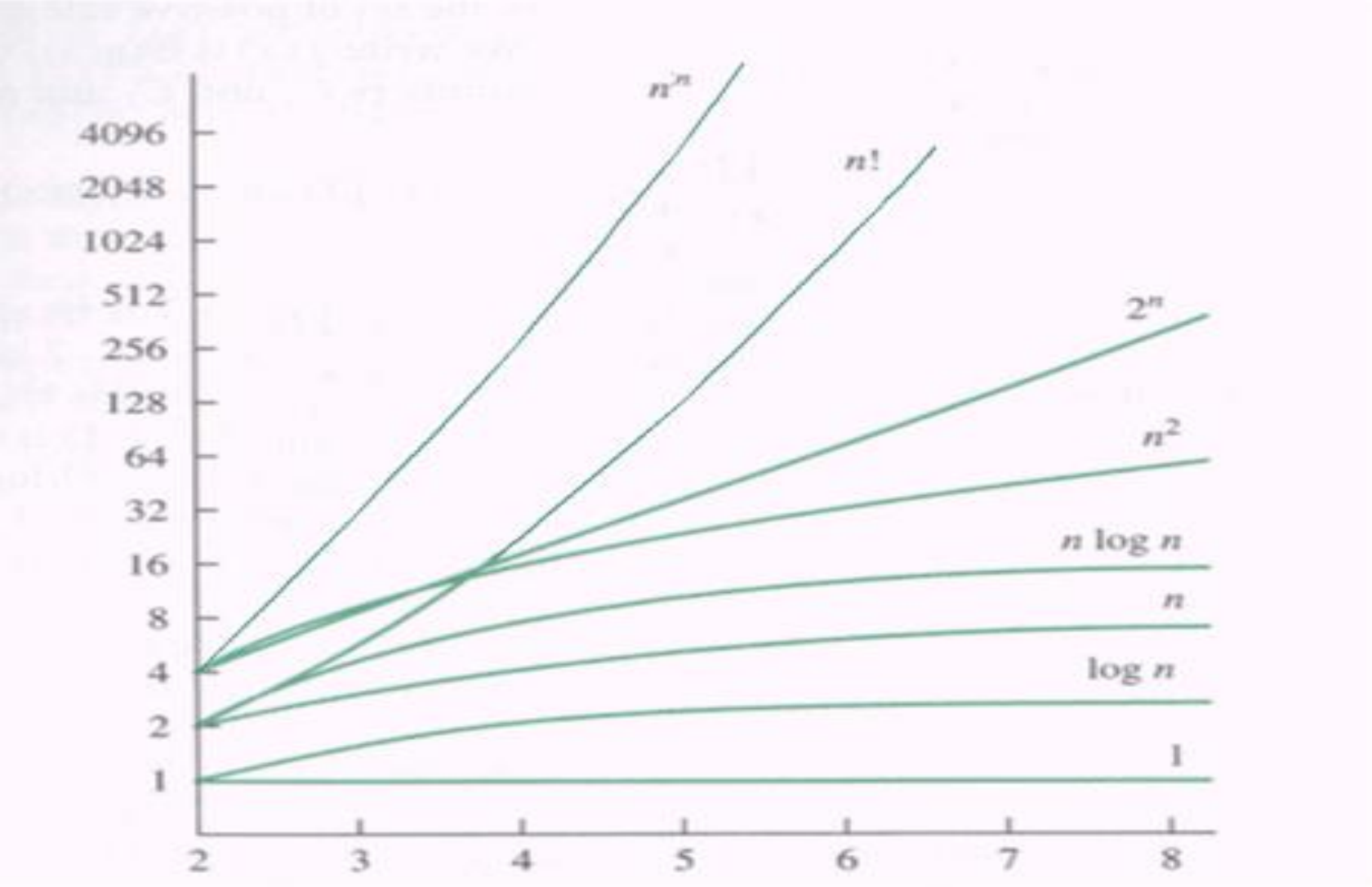
Question

- Find the Big O value for following fragment of code.

```
for i = 1 to n
    for j = 1 to i
        Print j
```

$O(n^2)$

Graphs of functions



n	$\log n$	n	$n\log n$	n^2	n^3	2^n
4	2	4	8	16	64	16
8	3	8	24	64	512	256
16	4	16	64	256	4,096	65,536
32	5	32	160	1,024	32,768	4,294,967,296
64	6	64	384	4,094	262,144	$1.84 * 10^{19}$
128	7	128	896	16,384	2,097,152	$3.40 * 10^{38}$
256	8	256	2,048	65,536	16,777,216	$1.15 * 10^{77}$
512	9	512	4,608	262,144	134,217,728	$1.34 * 10^{154}$
1024	10	1,024	10,240	1,048,576	1,073,741,824	$1.79 * 10^{308}$

Big O – Notation(Contd.)

- Find the Big O value for the following functions.

  (i) $T(n)= 3 +5n + 3n^2$

  (ii) $f(n)= 2^n + n^2 +8n +7$

  (iii) $T(n)= n + \log n +6$

Answers:

(i) $O(n^2)$

(ii) $O(2^n)$

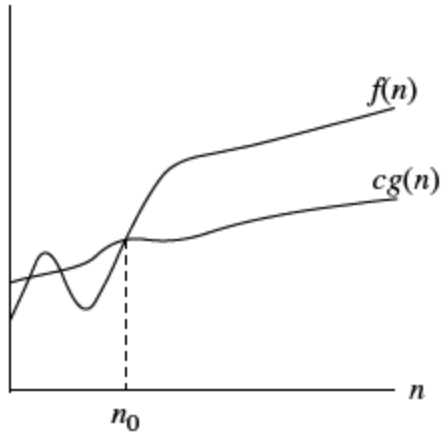
(iii) $O(n)$

Ω - Notation

- Provides the lower bound of the function.

Definition:

$\Omega(g(n)) = \{ f(n)$: there exist positive constants c and n_0 such that $0 \leq cg(n) \leq f(n)$ for all $n \geq n_o\}$



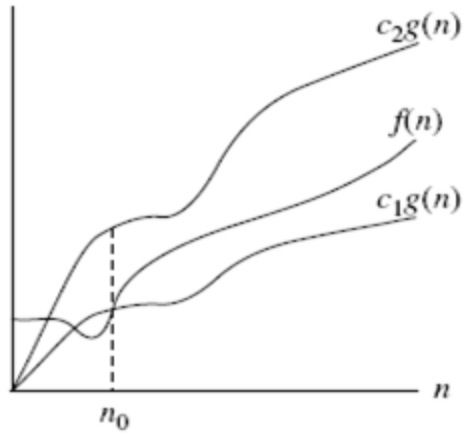
$g(n)$ is an ***asymptotic lower bound*** for $f(n)$.

Θ - Notation

- This is used when the function f can be bounded both from above and below by the same function g.

Definition:

$\Theta(g(n))$ ={ f(n): there exist positive constant c_1, c_2, and n_0 such that $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$ }



$g(n)$ is an ***asymptotically tight bound*** for $f(n)$.

References

- T.H. Cormen, C.E. Leiserson, R.L. Rivest, Clifford Stein Introduction to Algorithms,3rd Edition, MIT Press, 2009.